

CS1CA Computer Science

Data manipulation (2)

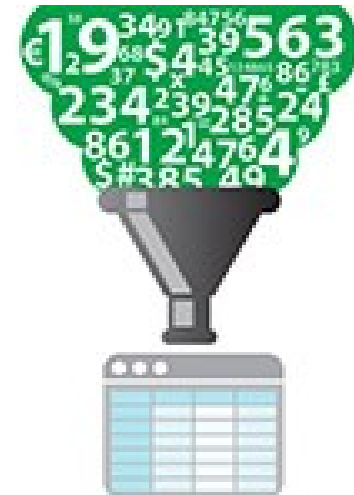
Week 3 lecture 2 of the autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133, Polly Vacher



Outline of the contents in the topic

- Computer architecture: introduction
- Memory organisation
- Instruction encoding and decoding
 - Introduction to a simple machine
 - A simple machine language
- Instruction execution cycle

In this lecture

- The following contents are delivered.
 - Instruction encoding and decoding
 - Introduction to a simple machine
 - A simple machine language
- The learning outcomes are listed below.
 - To be able to articulate the simple machine architecture
 - To be able to explain the process of instruction encoding and decoding with the given architecture.
 - To gain knowledge of machine language based on given architecture

Machine language: instructions

- When a computer runs a program, it takes our **instructions** (compiled from a program) to conduct tasks, such as editing a text file , or drawing a diagram.
- These instructions are meaningful to the CPU.
- **Terminology**
 - **Instructions**: encoded as bit patterns recognizable by the CPU
 - **Machine language**: a set of all instructions

Instruction types

Three main types of instructions

- **Data Transfer**: copy data from one location to another
 - Load R1, 10(R2) **meaning?**
- **Arithmetic/Logic**: use existing bit patterns to compute a new bit pattern
 - Add R3, R1, R2 **meaning?**
- **Control**: direct the execution of a program
 - Bnez R1, 10 **meaning?**

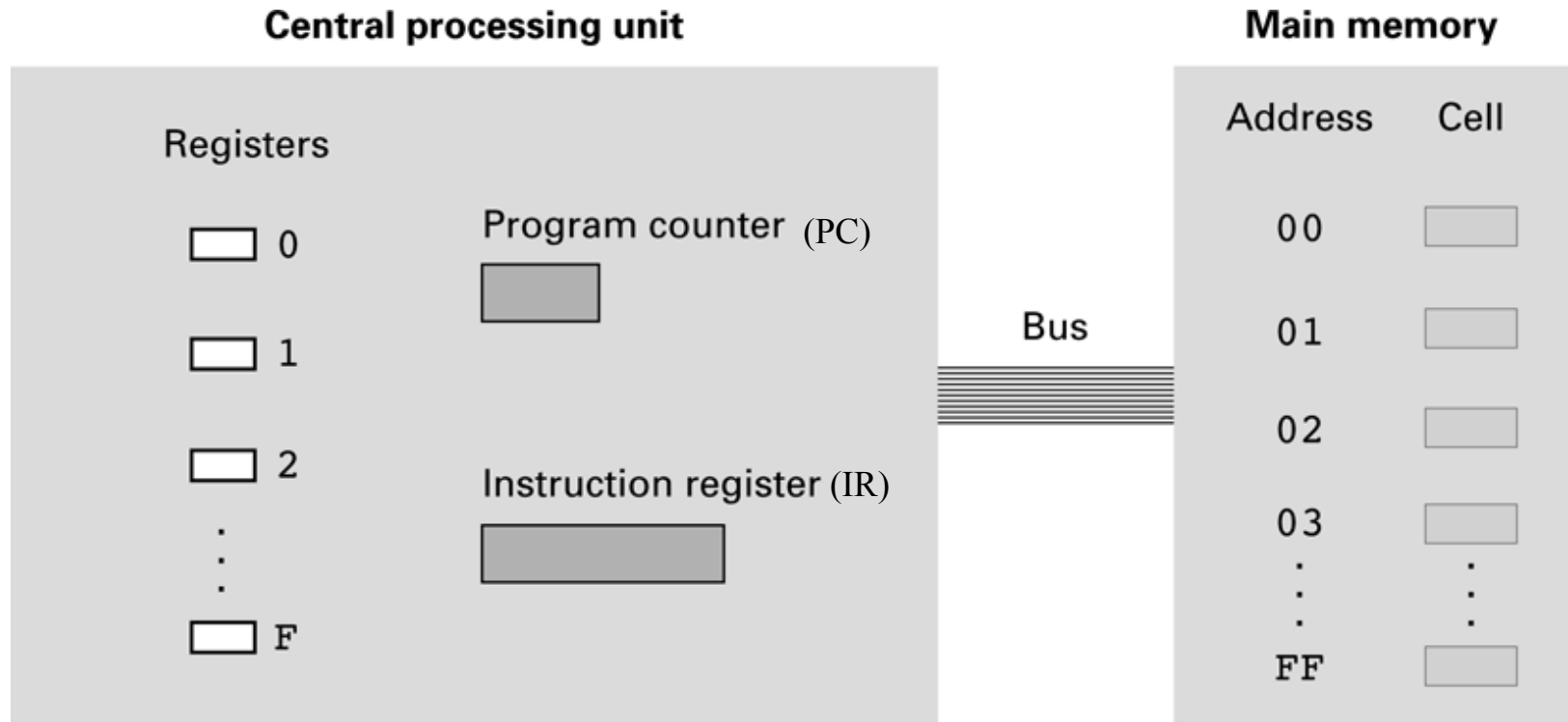
Instruction encoding

- Instruction encoding is a process in which a meaningful operation is encoded by **a binary representation** with designated fields.
- Different instruction sets may have different arrangements of these fields. However, the following items should normally be included in an instruction.
 - **Opcode**: specifies the operation, such as load, add,
 - **Operand**: gives more detailed information about the operation as “data” to be operated.
 - **Addressing mode**: shows where to fetch the data to be operated.

Introduction: architecture and machine language

- In order to design a machine language, it is **necessary** to understand its architecture.
- Machine language, in many cases, is also called **assembly language**.
- In assembly language, each code corresponds to an instruction (binary combination).
- Depending on the machine's architecture and its usage, we can start to design instructions and assign bits to different fields.

A simple architecture: registers and memory



A simple
architecture

- 16 GPRs (General Purpose Registers): 8 bits
- 2 SPRs: 8-bits PC and 16-bits IR
- 256 memory cells: each cell contains 8-bits
- Bus: 8 bits

Design instructions based on the given architecture

- 16 general purpose registers: 0, 1, 2, ..., F. This requires 4 bits to identify registers in an instruction.
- Each GPR register is one byte long (8 bits).
- 8 bits PC (for memory address) and 16 bits IR (for instruction)
- 256 cells in the main memory. This requires 8 bits to address them from 00 to FF.
- Fixed length of 16 bits for all instructions
 - 4-bits for op-code, and 12-bits for operands.
- 8 bits floating point (note: different from the IEEE 754 floating point format)

Example: Encoded instructions and meanings

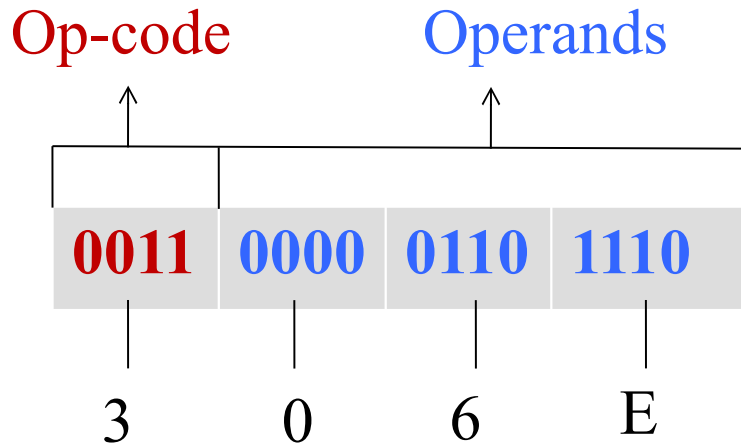
Encoded instructions	Translation
156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
C000	Halt.

In practice, we use hexadecimal to represent encoded binary instructions.

This is purely for the purpose of convenience because it uses fewer digits than binary.

Example: decompose instruction 306E

- 306E => 0011 0000 0110 1110
- It is a 16-bit instruction.
- There are two fields in the instruction
 - Opcode (addressing mode is included in this field)
 - Operands (data to be operated)



Actual bit pattern (16 bits)

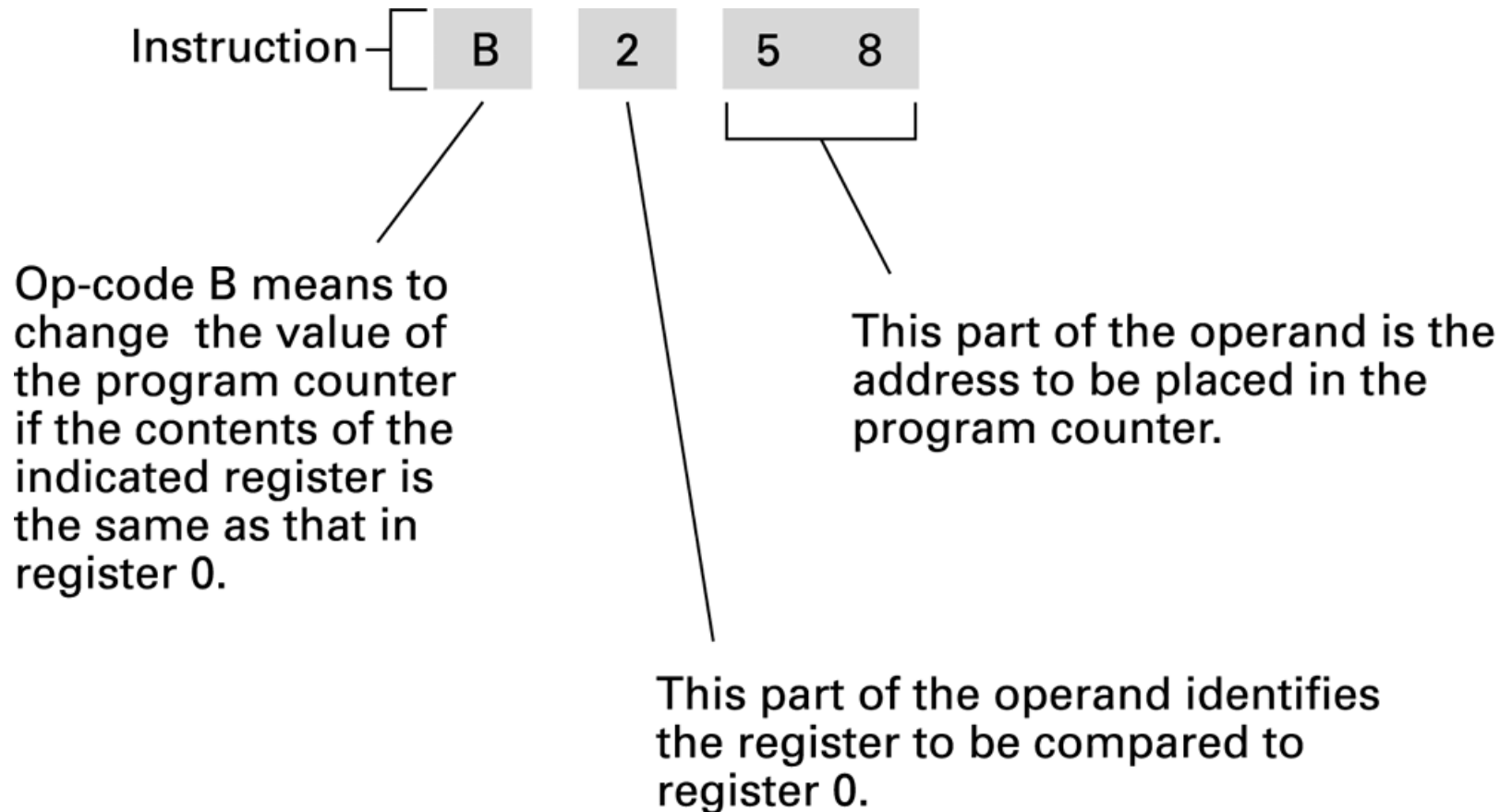
Hexadecimal form (4 digits)

Instruction decoding

- Decoding is to interpret the bits involved in the instruction and let it make sense to the CPU and programmer.
- For example: instruction **306E**
 - $0x3 \Rightarrow 0011$: **Op-code 3** means to store the content of a register in a memory cell.
 - $0x0 \Rightarrow 0000$: **Operand 0** identifies the register (R0) whose content is to be stored.
 - $0x6E \Rightarrow 0110\ 1110$: **Operand 6E** identifies the address of the memory cell where the data in R0 is stored.

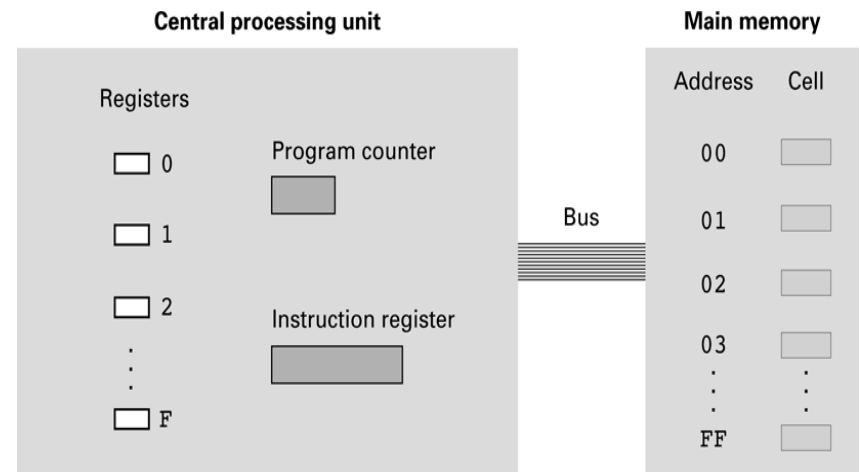
Another example of instruction decoding

- Decode the instruction B258



Introduction: a simple machine language

- Based on the simple machine introduced, we now look at the machine language designed for the architecture.
- The instruction set is simple. There are only 12 instructions to conduct operations in the machine.
 - **Data transfer**: Load, Store and Move
 - **Arithmetic/logic**: Add, Or, And, Rotate
 - **Control flow**: Jump
 - Halt (indicating the program ends)



The instruction set

Op-code	Operand	Description
1	RXY	LOAD reg. R from cell XY.
2	RXY	LOAD reg. R with XY.
3	RXY	STORE reg. R at cell XY.
4	0RS	MOVE reg. R to reg. S.
5	RST	ADD S and T into R. (2's comp.)
6	RST	ADD S and T into R. (floating pt.)

The instruction set (continues...)

Op-code	Operand	Description
7	RST	OR S and T into R.
8	RST	AND S and T into R.
9	RST	XOR S and T into R.
A	R0X	ROTATE reg. R right X times.
B	RXY	JUMP to XY if R = reg. 0.
C	000	HALT.

For the detailed instruction set information, please refer to the appendix of the self-assessed coursework file in the Blackboard (BB).

Meanings of letters in instructions

- There are 12 op-codes numbered as 1...C in hexadecimal
- For the letters R, S, and T, each represents a register identifier in 4 bits as a hexadecimal digit.
- The letters X and Y represent variables in hexadecimal, and XY is a memory address in 8 bits or an 8-bit number.
- Example instruction: 0xA403

Examples of Instructions

Op - code	Operands	Operation	Meaning
1	4A3	load	Load the content of the memory cell located at address A3 and place it in Register 4
2	1A3	load	Load the value A3 to register 1
4	0A4	move	Copy contents in register A to register 4
5	726	add	Add the binary value in register 2 to the binary value in register 6 and place the result in register 7
7	CB4	or	OR the contents of registers B and 4, and place the result in register C
A	403	rotate	Rotate the content of register 4 for 1 bit to the right 3 times in a circular fashion
B	43C	jump/branch	First compare the content of register 4 with the content of register 0. If the two were equal, 3C will be placed in PC to fetch the next instruction; if not, nothing will be done, and the execution continues.

End of this lecture – Questions and exercises

- Describe the instruction encoding and decoding process.
 - What are the three types of instructions?
- Articulate the simple machine architecture and its language.
 - How many memory cells are there in the machine?
 - How many registers are there?
 - How many instructions used in the machine?
- Explore additional resources to find out different machine architectures and machine languages, such as ARM architecture or DLX architecture.